Fast Tokenizers in the QA Pipeline

Now we will dive deeper into the question-answering pipeline and see how to use offsets to extract the answer to a question from the context—similar to what we did with grouped entities in the previous section. Then we will explore how to handle very long contexts that get truncated. If you're not interested in the question-answering task, you can skip this section.

Using the Question-Answering Pipeline

We can use the question-answering pipeline like this:

```
from transformers import pipeline

question_answerer = pipeline("question-answering")
context = """
Transformers is backed by three popular deep learning libraries—Jax, PyTorch, and
TensorFlow—and provides seamless integration between them.
You can train a model in one library and easily run inference in another.
"""
question = "Which deep learning libraries support Transformers?"
question_answerer(question=question, context=context)
```

```
{'score': 0.97773,
 'start': 78,
 'end': 105,
 'answer': 'Jax, PyTorch and TensorFlow'}
```

Unlike other pipelines—which truncate text when it exceeds the model's maximum length and may miss information at the end—this pipeline can handle very long contexts and still find the correct answer even if it appears near the end of the document.

```
long_context = """
Transformers: State of the Art NLP

Transformers provides thousands of pretrained models that can perform tasks such as
classification, information extraction,
question answering, summarization, translation, and text generation in over 100 languages.

Its goal is to make advanced NLP easy to use for everyone.
```

Transformers provides APIs to quickly download and use pretrained models, fine-tune them on your own datasets, and share them on the model hub.

Why use Transformers?

1. Easy-to-use modern models:
  - High performance on NLU and NLG tasks
  - Easy access for students and practitioners
  - Minimal abstraction—only three classes to learn
  - Unified API for all pretrained models
  - Lower compute cost and carbon footprint

2. Researchers can share trained models:
  - Reduces the need for retraining
  - Saves time and cost
  - 10,000+ pretrained models in 100+ languages

3. Choose the right framework for each stage of a model:
  - Train models in just 3 lines of code
  - Easily switch between TF2.0 and PyTorch
  - Use the best framework for training, evaluation, and production

4. Easy customization:
  - Examples available for each architecture
  - Transparent model internals
  - Model files usable outside the library

Transformers is backed by Jax, PyTorch, and TensorFlow.
"""
question_answerer(question=question, context=long_context)

{'score': 0.97149,
 'start': 1892,
 'end': 1919,
 'answer': 'Jax, PyTorch and TensorFlow'}

Using a Model for Question Answering

Like any other pipeline, we first tokenize the input and then pass it through the model. The default checkpoint used for the question-answering pipeline is `distilbert-base-cased-distilled-squad`. The “squad” part comes from the dataset used to fine-tune the model. We will discuss the SQuAD dataset in more detail in Chapter 7.

```
from transformers import AutoTokenizer, AutoModelForQuestionAnswering

model_checkpoint = "distilbert-base-cased-distilled-squad"
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
model = AutoModelForQuestionAnswering.from_pretrained(model_checkpoint)

inputs = tokenizer(question, context, return_tensors="pt")
outputs = model(**inputs)
```

We tokenize the question and the context as a pair, with the question placed first.

An example of tokenizing question and context

Question-answering models work a bit differently from the models we've seen so far. As illustrated in the example above, the model is trained to predict the index of the token where the answer starts (here, 21) and the index where the answer ends (here, 24).

That's why these models do not return a single logits tensor, but instead return two:

- one for the logits corresponding to the start token of the answer
- another for the logits corresponding to the end token of the answer

Since we have only one input containing 66 tokens, we get:

```
start_logits = outputs.start_logits
end_logits = outputs.end_logits
print(start_logits.shape, end_logits.shape)
```

```
torch.Size([1, 66]) torch.Size([1, 66])
```

To convert these logits into probabilities, we apply a softmax function. But before doing that, we must ensure that indices not belonging to the context are masked.

Our input format is:

[CLS] question [SEP] context [SEP]

So we need to mask the tokens of the question as well as the `[SEP]` token. However, we keep the `[CLS]` token unmasked, since some models use it to indicate that the answer is not present in the context.

Since we will apply softmax afterward, we can simply replace the logits we want to mask with a large negative number. Here we use `-10000`:

```
import torch

sequence_ids = inputs.sequence_ids()
# Mask everything except the context tokens
mask = [i != 1 for i in sequence_ids]
# Unmask the [CLS] token
mask[0] = False
mask = torch.tensor(mask)[None]

start_logits[mask] = -10000
end_logits[mask] = -10000
```

Now that the unwanted positions are properly masked, we can apply softmax:

```
start_probabilities = torch.nn.functional.softmax(start_logits, dim=-1)[0]
end_probabilities = torch.nn.functional.softmax(end_logits, dim=-1)[0]
```

At this stage, we could take the argmax of the start and end probabilities. However, this might result in a start index that is greater than the end index. So we need to be more careful.

We compute the probability for all possible pairs of `start_index` and `end_index` where `start_index <= end_index`, and then select the pair with the highest probability.

Assuming the events “the answer starts at `start_index`” and “the answer ends at `end_index`” are independent, the joint probability is:

```
start_probabilities[start_index] * end_probabilities[end_index]
```

So to compute all scores, we calculate all such products where `start_index <= end_index`.

First, compute all possible products:

```
scores = start_probabilities[:, None] * end_probabilities[None, :]
```

Next, mask out the cases where `start_index > end_index` by setting them to 0 (since all valid probabilities are positive). The `torch.triu()` function returns the upper triangular part of a 2D tensor, which conveniently performs this masking:

```
scores = torch.triu(scores)
```

Now we just need to find the index of the maximum value. Since PyTorch returns the index in the flattened tensor, we use `//` and `%` to recover the `start_index` and `end_index`:

```
max_index = scores.argmax().item()
start_index = max_index // scores.shape[1]
end_index = max_index % scores.shape[1]
print(scores[start_index, end_index])
```

We're not completely done yet, but we already have the correct score for the answer. You can verify this by comparing it with the earlier result:

```
0.97773
```

Exercise

Compute the start and end indices for the five most likely answers.

Now we have the `start_index` and `end_index` in terms of tokens. The next step is to convert them into character indices in the context. This is where offsets become very useful. We can use them just like in the token classification task:

```
inputs_with_offsets = tokenizer(question, context, return_offsets_mapping=True)
offsets = inputs_with_offsets["offset_mapping"]

start_char, _ = offsets[start_index]
_, end_char = offsets[end_index]
answer = context[start_char:end_char]
```

Finally, we format everything to get the result:

```
result = {
    "answer": answer,
    "start": start_char,
    "end": end_char,
    "score": scores[start_index, end_index],
}
print(result)

{'answer': 'Jax, PyTorch and TensorFlow',
```

'start': 78,
'end': 105,
'score': 0.97773}

Handling Long Contexts

If we try to tokenize the question and the long context used in the previous example, the number of tokens will exceed the maximum limit of 384 used by the question-answering pipeline:

```
inputs = tokenizer(question, long_context)
print(len(inputs["input_ids"]))
```

461

So we need to truncate the input to that maximum length. There are several ways to do this, but we do not want to truncate the question, only the context. Since the context is the second sentence, we use the `"only_second"` truncation strategy.

The problem is that the answer to the question may not be present in the truncated context. In this example, we chose a question whose answer appears near the end of the context, and after truncation that answer is no longer present:

```
inputs = tokenizer(question, long_context, max_length=384, truncation="only_second")
print(tokenizer.decode(inputs["input_ids"]))
```

"""
[CLS] Which deep learning libraries back [UNK] Transformers? [SEP] [UNK] Transformers : State of the Art NLP

[UNK] Transformers provides thousands of pretrained models to perform tasks on texts such as classification, information extraction,
question answering, summarization, translation, text generation and more in over 100 languages.
Its aim is to make cutting-edge NLP easier to use for everyone.

[UNK] Transformers provides APIs to quickly download and use those pretrained models on a given text, fine-tune them on your own datasets and
then share them with the community on our model hub. At the same time, each python module defining an architecture is fully standalone and
can be modified to enable quick research experiments.

Why should I use transformers?

1. Easy-to-use state-of-the-art models:
  - High performance on NLU and NLG tasks.
  - Low barrier to entry for educators and practitioners.
  - Few user-facing abstractions with just three classes to learn.
  - A unified API for using all our pretrained models.
  - Lower compute costs, smaller carbon footprint:

2. Researchers can share trained models instead of always retraining.
  - Practitioners can reduce compute time and production costs.
  - Dozens of architectures with over 10,000 pretrained models, some in more than 100 languages.

3. Choose the right framework for every part of a model's lifetime:
  - Train state-of-the-art models in 3 lines of code.
  - Move a single model between TF2.0/PyTorch frameworks at will.
  - Seamlessly pick the right framework for training, evaluation and production.

4. Easily customize a model or an example to your needs:
  - We provide examples for each architecture to reproduce the results published by its original authors.
  - Model internal [SEP]
"""

This means the model will have difficulty finding the correct answer.

To solve this, the question-answering pipeline allows us to split the context into smaller chunks while specifying a maximum length. To avoid splitting the context at exactly the wrong place and losing the answer, it also keeps some overlap between chunks.

We can have the tokenizer do this for us by setting `return_overflowing_tokens=True`. The amount of overlap can be controlled with the `stride` argument. Here is an example using a short sentence:

```
sentence = "This sentence is not too long but we are going to split it anyway."
inputs = tokenizer(
    sentence, truncation=True, return_overflowing_tokens=True, max_length=6, stride=2
)

for ids in inputs["input_ids"]:
    print(tokenizer.decode(ids))
```

```
'[CLS] This sentence is not [SEP]'
```

'[CLS] is not too long [SEP]'
'[CLS] too long but we [SEP]'
'[CLS] but we are going [SEP]'
'[CLS] are going to split [SEP]'
'[CLS] to split it anyway [SEP]'
'[CLS] it anyway. [SEP]'

As we can see, the sentence has been split into chunks so that each entry in `inputs["input_ids"]` contains at most 6 tokens. We would need to add padding if we wanted the last entry to have the same length as the others. There is also an overlap of 2 tokens between consecutive entries.

Now let's look more closely at the tokenization result:

print(inputs.keys())

dict_keys(['input_ids', 'attention_mask', 'overflow_to_sample_mapping'])

As expected, we get input IDs and an attention mask. The last key, `overflow_to_sample_mapping`, is a map that tells us which result corresponds to which sentence. Here, all 7 results come from the single sentence we passed to the tokenizer:

print(inputs["overflow_to_sample_mapping"])

[0, 0, 0, 0, 0, 0, 0]

This becomes even more useful when tokenizing multiple sentences together. For example:

```
sentences = [
    "This sentence is not too long but we are going to split it anyway.",
    "This sentence is shorter but will still get split.",
]
inputs = tokenizer(
    sentences, truncation=True, return_overflowing_tokens=True, max_length=6, stride=2
)

print(inputs["overflow_to_sample_mapping"])
```

The result is:

[0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1]

This means the first sentence was split into 7 chunks as before, and the next 4 chunks come from the second sentence.

Now let's go back to our long context. As mentioned earlier, the question-answering pipeline uses `max_length=384` and `stride=128` by default. This matches the way the model was fine-tuned. You can change these values by passing `max_seq_len` and `stride` when calling the pipeline.

So while tokenizing, we use those parameters. We also add padding so that all samples have the same length and can be converted into tensors. We also request offsets:

```
inputs = tokenizer(
    question,
    long_context,
    stride=128,
    max_length=384,
    padding="longest",
    truncation="only_second",
    return_overflowing_tokens=True,
    return_offsets_mapping=True,
)
```

These `inputs` contain the input IDs, attention masks, offsets, and `overflow_to_sample_mapping`. Since the last two are not model parameters, we remove them before converting to tensors. We do not need to keep the mapping here:

```
_ = inputs.pop("overflow_to_sample_mapping")
offsets = inputs.pop("offset_mapping")

inputs = inputs.convert_to_tensors("pt")
print(inputs["input_ids"].shape)
```

```
torch.Size([2, 384])
```

Our long context was split into two parts. So after passing it through the model, we get two sets of start logits and end logits:

```
outputs = model(**inputs)

start_logits = outputs.start_logits
end_logits = outputs.end_logits
```

```
print(start_logits.shape, end_logits.shape)
```

```
torch.Size([2, 384]) torch.Size([2, 384])
```

As before, before applying softmax we mask the tokens that are not part of the context. We also mask the padding tokens, as indicated by the attention mask:

```
sequence_ids = inputs.sequence_ids()
# Mask everything except the context tokens
mask = [i != 1 for i in sequence_ids]
# Unmask the [CLS] token
mask[0] = False
# Mask all [PAD] tokens
mask = torch.logical_or(torch.tensor(mask)[None], (inputs["attention_mask"] == 0))

start_logits[mask] = -10000
end_logits[mask] = -10000
```

Then we can convert the logits into probabilities using softmax:

```
start_probabilities = torch.nn.functional.softmax(start_logits, dim=-1)
end_probabilities = torch.nn.functional.softmax(end_logits, dim=-1)
```

The next step is similar to what we did for the short context, except now we do it for each chunk. We compute a score for every possible answer span, then select the span with the best score:

```
candidates = []
for start_probs, end_probs in zip(start_probabilities, end_probabilities):
    scores = start_probs[:, None] * end_probs[None, :]
    idx = torch.triu(scores).argmax().item()

    start_idx = idx // scores.shape[1]
    end_idx = idx % scores.shape[1]
    score = scores[start_idx, end_idx].item()
    candidates.append((start_idx, end_idx, score))

print(candidates)
```

```
[(0, 18, 0.33867), (173, 184, 0.97149)]
```

These two candidates are the best answers the model found in each chunk. The model is much more confident that the correct answer is in the second chunk, which is a good sign.

Now we just need to map these token spans back to character spans in the context. In practice, we only need the second one to get the answer, but it is still interesting to see what the model picked in the first chunk.

Exercise

Modify the code above so that it returns the scores and spans of the five most likely answers across the whole context, not separately for each chunk.

The `offsets` we retrieved earlier is actually a list of offsets, with one list for each chunk:

```
for candidate, offset in zip(candidates, offsets):
    start_token, end_token, score = candidate
    start_char, _ = offset[start_token]
    _, end_char = offset[end_token]
    answer = long_context[start_char:end_char]
    result = {"answer": answer, "start": start_char, "end": end_char, "score": score}
    print(result)
```

```
{'answer': '\nTransformers: State of the Art NLP', 'start': 0, 'end': 37, 'score': 0.33867}
{'answer': 'Jax, PyTorch and TensorFlow', 'start': 1892, 'end': 1919, 'score': 0.97149}
```